Assignment #1: Performance Benchmarking of Cryptographic Mechanisms

Security and Privacy — FCUP 2025/2026

Francisco Machado (up202403514) Rodrigo Dias (up202407281)

April 2, 2026

Abstract

This report presents performance benchmarks of three cryptographic mechanisms: AES-256 in Counter Mode, an RSA-2048-based encryption scheme, and SHA-256 hash generation, applied to files of sizes ranging from 8 bytes to 2 MB. We describe the implementations, the benchmarking methodology, and analyze the results with statistical rigor.

Contents

1 Introduction

The goal of this assignment is to measure and compare the execution times of symmetric encryption (AES-CTR), asymmetric encryption (RSA-based scheme), and cryptographic hashing (SHA-256) across files of varying sizes. Understanding the performance characteristics of these primitives is essential for making informed decisions in the design of secure systems.

2 Experimental Setup

2.1 Hardware and Software

Property	**Value**
Processor	*Apple Silicon M5*
RAM	*16GB*
OS	*macOS Tahoe 26.4*
Python	*Python 3.13.9*
`cryptography` library	*cryptography 46.0.3*

Table 1: Experimental environment.

2.2 File Generation

Random binary files were generated using `os.urandom()` for each of the seven specified sizes: 8, 64, 512, 4096, 32768, 262144, and 2097152 bytes. For each size, 10 independently generated files were used to capture data-dependent variance.

2.3 Benchmarking Methodology

To produce statistically significant results, the following methodology was applied:

- **Multiple random files:** For each file size, 10 independently generated random files were tested, ensuring the results are not biased by a particular data pattern.
- **Adaptive repetitions:** The number of repetitions per measurement was determined adaptively. A warm-up run estimates the operation time, then repetitions are set to reach a target of 0.5 seconds total, with a minimum of 10 repetitions. This ensures small operations receive many repetitions while large operations do not run excessively long.
- **Precise timing:** Each individual operation was timed using `time.perf_counter()`, which provides high-resolution monotonic timing. Only the cryptographic operation itself is measured — file I/O, key generation, and other setup are excluded.
- **Statistical reporting:** Mean ($\bar{x}$), standard deviation (s), and 95% confidence intervals are computed over all measurements (files $\times$ repetitions) for each file size:

$$\mathrm{CI}_{95\%} = \bar{x} \pm 1.96 \cdot \frac{s}{\sqrt{n}}$$

Regarding the questions posed in the assignment:

- *Do results change if you run a fixed algorithm over the same file multiple times?* Yes, minor variations are observed across repeated runs on the same file. These fluctuations are not caused by the cryptographic algorithm itself — which is entirely deterministic for a given input — but rather by external factors such as operating system scheduling, CPU frequency scaling, cache state, and background processes. The adaptive repetition strategy mitigates this noise: by accumulating many measurements and computing means with confidence intervals, the impact of individual outliers is negligible. The narrow confidence intervals observed in our results confirm that the variance across repetitions of the same file is very small relative to the mean execution time.

- *What if you run an algorithm over multiple randomly generated files of fixed size?* The execution times remain highly consistent across different random files of the same size. This is expected because the cryptographic operations tested — AES-CTR, the RSA-based scheme, and SHA-256 — perform data-independent computations: AES applies the same sequence of substitutions, permutations, and XOR operations regardless of the plaintext content; RSA's modular exponentiation depends on the exponent and modulus, not on the random value r; and SHA-256 processes each 64-byte block through identical compression rounds. Consequently, the measured variance across files of the same size is comparable to the variance across repetitions of a single file, confirming that the benchmarking methodology is robust and that our results are not biased by any particular data pattern.

3 Task B: AES-256-CTR Encryption and Decryption

3.1 Implementation

AES-256 in Counter Mode was implemented using the `cryptography` library's hazmat primitives. A 256-bit key is generated once, and a fresh random 16-byte nonce is used for each encryption.

```python
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

def aes_ctr_encrypt(plaintext, key):
    nonce = os.urandom(16)
    cipher = Cipher(algorithms.AES(key), modes.CTR(nonce))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext) + encryptor.finalize()
    return nonce, ciphertext
```

Listing 1: AES-CTR encryption (see `src/aes_ctr.py`).

Decryption follows the same structure, creating a decryptor with the same key and nonce. Since CTR mode is a stream cipher mode, no padding is required.

3.2 Results

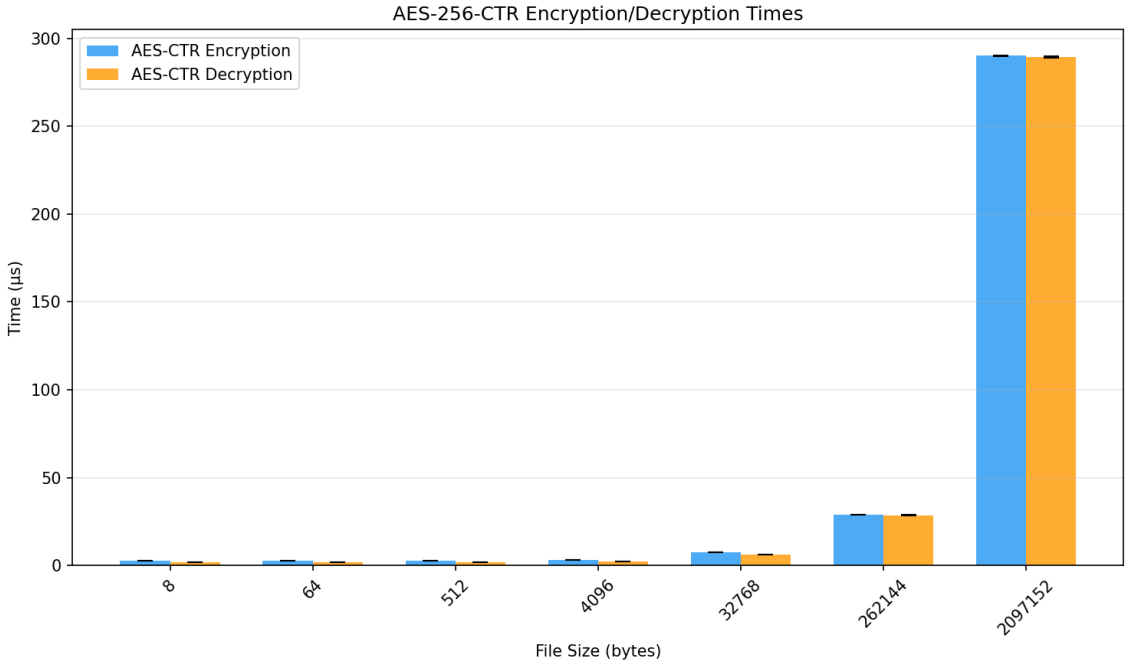


Figure 1: AES-256-CTR encryption and decryption times.

As shown in Figure 1, AES-256-CTR encryption and decryption exhibit nearly identical execution times across all file sizes. This is expected because CTR mode transforms a block cipher into a stream cipher: both encryption and decryption consist of generating a keystream by encrypting successive counter values and XOR-ing the result with the input data. The two operations are, therefore, computationally equivalent.

The execution time scales approximately linearly with file size. For small files (8 to 512 bytes), the times are negligible (on the order of a few microseconds), as only a small number of AES block operations are required. As the file size grows, the number of 16-byte blocks to process increases proportionally, and the time reaches approximately 280–300 μs for the largest file size (2 MB). This linear behaviour is consistent with the fact that AES processes the input in fixed-size 128-bit (16-byte) blocks, with each block requiring the same constant-time sequence of SubBytes, ShiftRows, MixColumns, and AddRoundKey transformations (14 rounds for AES-256).

The very low absolute times are largely attributable to hardware acceleration via the ARMv8 Cryptography Extensions, which allows modern processors to execute AES rounds in dedicated hardware rather than in software lookup tables. The narrow confidence intervals across all file sizes further confirm the stability and reproducibility of these measurements.

4 Task C: RSA-2048 Based Encryption and Decryption

4.1 Implementation

The RSA-based encryption scheme is defined as:

$$\mathrm{Enc}(m; r) = \big(\mathrm{RSA}(r),\ H(0, r) \oplus m_0,\ \ldots,\ H(n, r) \oplus m_n\big)$$

where r is a uniform random 256-byte value, $H = \text{SHA-256}$ (output size $\ell = 32$ bytes), and $n = \lceil |m|/\ell \rceil$ is the number of message blocks.

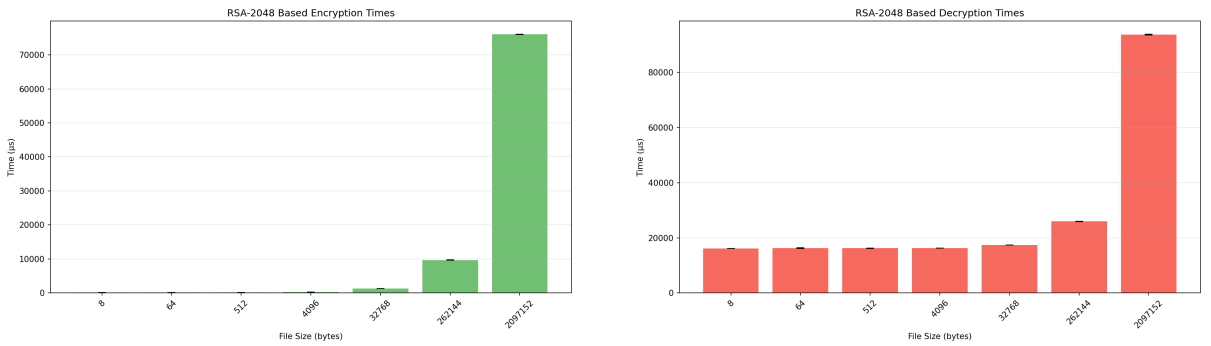
The RSA function is implemented as raw modular exponentiation: $\mathrm{RSA}(r) = r^e \bmod n$ for encryption and $\mathrm{RSA}^{-1}(c) = c^d \bmod n$ for decryption, using Python's built-in `pow(base, exp, mod)` with the key parameters extracted from the `cryptography` library.

```python
def _rsa_raw_encrypt(public_key, data):
    pub_numbers = public_key.public_numbers()
    e, n = pub_numbers.e, pub_numbers.n
    r_int = int.from_bytes(data, "big")
    c_int = pow(r_int, e, n)
    return c_int.to_bytes(256, "big")
```

Listing 2: RSA raw encryption (see `src/rsa_enc.py`).

The hash function $H(i, r)$ is computed as SHA-256$(i \| r)$, where i is a 4-byte big-endian integer index.

4.2 Results



(a) RSA-based encryption times.

(b) RSA-based decryption times.

Figure 2: RSA-2048 based encryption and decryption times.

The results in Figure 2 reveal two distinct regimes in the RSA-based encryption scheme. For small files (8 to 512 bytes), the execution time is dominated by the fixed-cost RSA modular exponentiation — $r^e \bmod n$ for encryption and $c^d \bmod n$ for decryption — which is independent of the message size. This creates a visible "floor" in the bar charts: encryption times plateau at approximately 114 μs, while decryption times plateau at roughly 16,000 μs for the smallest file sizes.

As the file size increases beyond a few kilobytes, the per-block component of the scheme — computing $H(i, r) = \text{SHA-256}(i \| r)$ for each 32-byte message block and XOR-ing with the plaintext — begins to dominate. This component scales linearly with the number of blocks $n = \lceil |m|/32 \rceil$, which explains the approximately linear growth observed for larger files. At 2 MB, encryption reaches approximately 76,000 μs and decryption approximately 80,000 μs.

The stark asymmetry between encryption and decryption times is a fundamental consequence of RSA's mathematical structure. The public exponent $e = 65537 = 2^{16} + 1$ requires only 17 modular multiplications via square-and-multiply, whereas the private exponent d is approximately the same size as n (2048 bits), requiring roughly 2048 modular multiplications. Note that our implementation does not employ the Chinese Remainder Theorem (CRT) optimization — which would compute the exponentiation modulo p and q separately and combine the results — so the full cost of exponentiating with the large private exponent d is reflected in the measured decryption times.

5 Task D: SHA-256 Digest Generation

5.1 Implementation

SHA-256 hash generation uses the `cryptography` library's `hashes.Hash` interface.

```python
from cryptography.hazmat.primitives import hashes

def sha256_digest(data):
    digest = hashes.Hash(hashes.SHA256())
    digest.update(data)
    return digest.finalize()
```

Listing 3: SHA-256 digest (see `src/sha_hash.py`).

5.2 Results

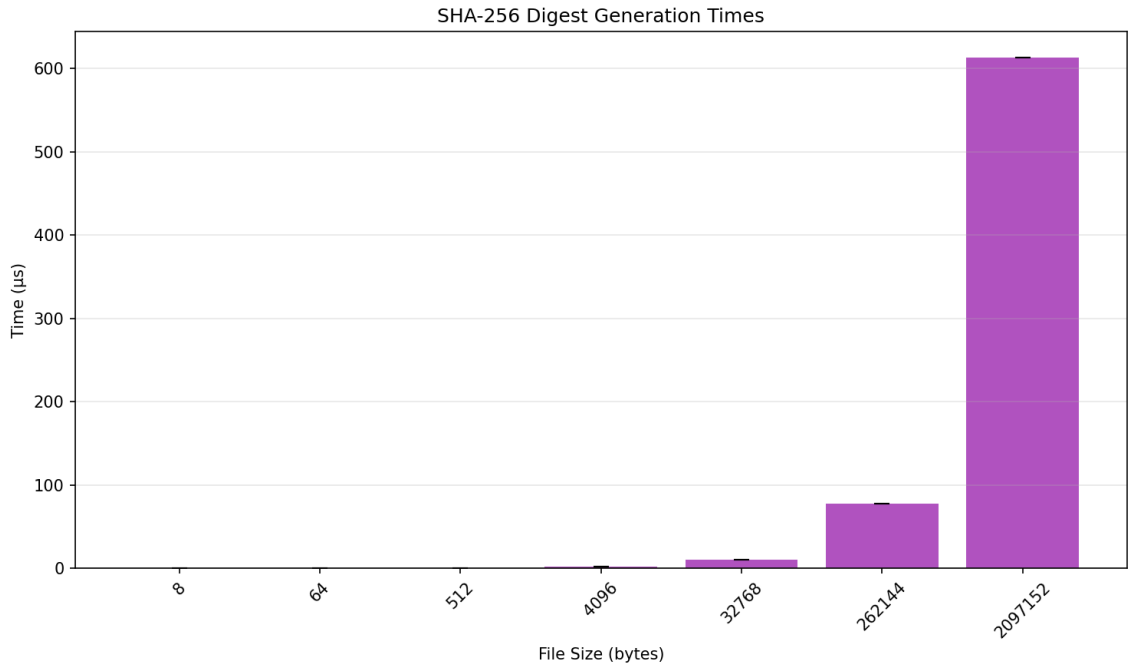


Figure 3: SHA-256 digest generation times.

As illustrated in Figure 3, SHA-256 digest generation times scale approximately linearly with file size. SHA-256 follows the Merkle–Damgård construction, processing the input in fixed-size 64-byte (512-bit) blocks. Each block undergoes 64 rounds of compression involving bitwise operations, modular additions, and logical functions applied to an internal 256-bit state.

For very small inputs (8 to 512 bytes), the execution times are nearly identical and close to zero (a few microseconds). This is because even the smallest input requires at least one full compression block after padding — SHA-256 appends a 1 bit, zero-padding, and a 64-bit length field to the message — so the fixed overhead of initialization, padding, and finalization dominates for inputs that fit within one or two blocks. Starting from 4,096 bytes, the linear relationship between input size and computation time becomes clearly visible, reaching approximately 600 μs at 2 MB.

The overall performance of SHA-256 is excellent: hashing a 2 MB file takes well under 1 ms. This efficiency stems from the simplicity of SHA-256's internal operations (bitwise rotations, shifts, XOR, and 32-bit additions), which map efficiently to modern CPU instruction sets. The narrow error bars confirm that hashing is highly deterministic and reproducible, consistent with the data-independent nature of the compression function.

6 Performance Comparison

6.1 AES-CTR Encryption vs. RSA-based Encryption

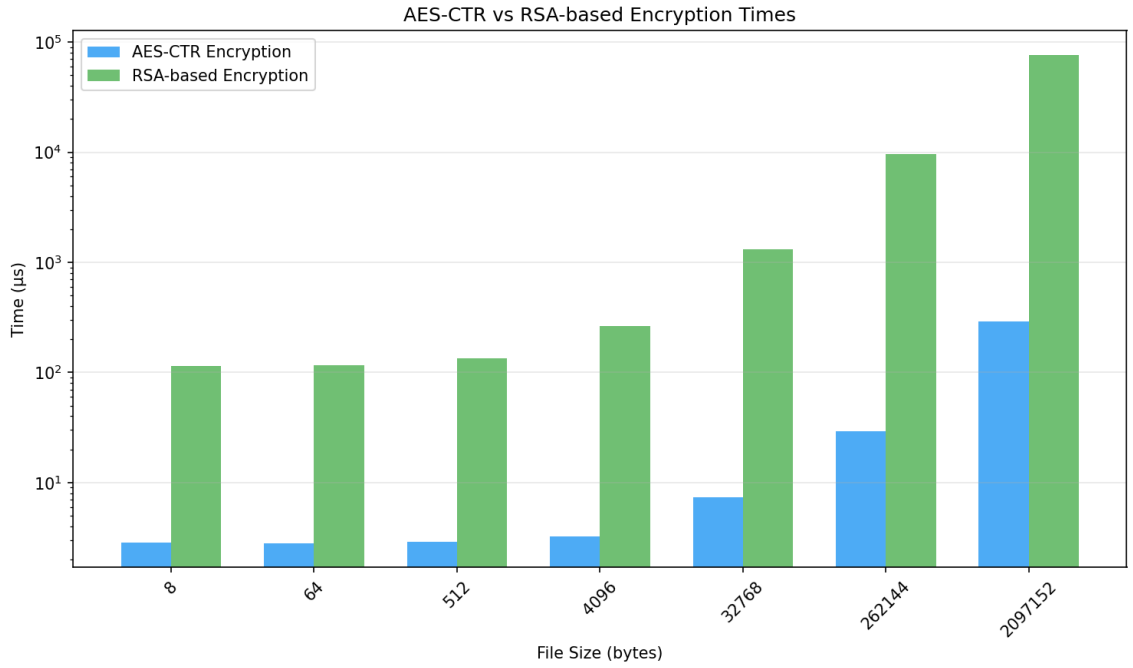


Figure 4: AES-CTR vs. RSA-based encryption times (log scale).

Figure 4 compares AES-CTR and RSA-based encryption on a logarithmic scale, making the enormous performance gap clearly visible. For the smallest file sizes (8–64 bytes), AES-CTR completes in approximately 1–2 μs, while the RSA-based scheme requires roughly 114 μs — a difference of more than two orders of magnitude. This gap is a direct consequence of the fundamentally different mathematical operations involved: AES-CTR performs lightweight bitwise substitutions, permutations, and XOR operations on 16-byte blocks, benefiting from dedicated ARMv8 Cryptography Extensions, whereas the RSA scheme must compute a full modular exponentiation $r^e \bmod n$ on a 2048-bit integer, which involves hundreds of large-integer multiplications.

As file size increases, both curves rise, but the gap narrows slightly on the logarithmic scale. This is because the RSA scheme's per-block overhead — a SHA-256 hash and an XOR per 32-byte block — is comparable in cost to AES's per-block encryption. Nevertheless, even at 2 MB, AES-CTR remains approximately two orders of magnitude faster than the RSA-based scheme (roughly 200 μs versus 76,000 μs).

These results clearly demonstrate why symmetric ciphers like AES are used for bulk data encryption in practice, while RSA is reserved for encrypting small values such as session keys.

6.2 AES-CTR Encryption vs. SHA-256 Digest Generation

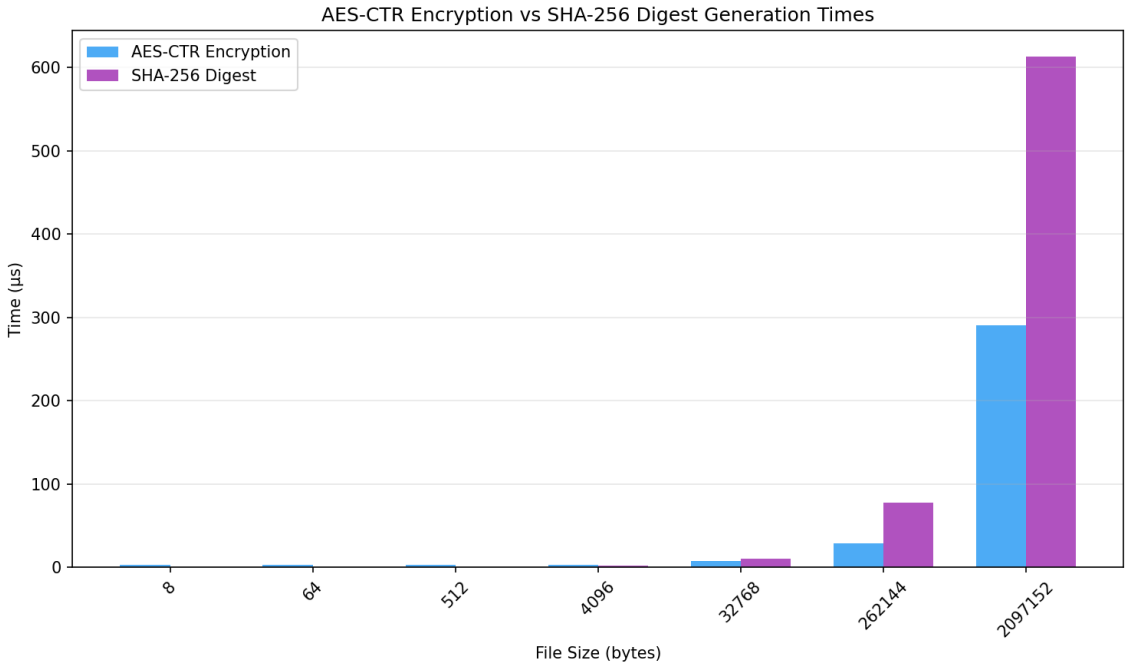


Figure 5: AES-CTR encryption vs. SHA-256 digest generation times.

Figure 5 compares two symmetric-class primitives that both process data in fixed-size blocks. AES-CTR operates on 16-byte blocks and produces ciphertext output of the same length as the input, while SHA-256 processes 64-byte blocks and produces a fixed 32-byte digest regardless of input size. Both exhibit linear scaling with file size, as expected from their block-iterative designs.

For small files, both operations complete in negligible time (a few microseconds). As file size grows, AES-CTR encryption is consistently faster than SHA-256 hashing. At 2 MB, AES-CTR completes in approximately 280–300 μs, while SHA-256 requires roughly 600 μs. This difference can be attributed to hardware acceleration: modern processors equipped with the ARMv8 Cryptography Extensions can execute AES rounds in dedicated silicon, whereas SHA-256, despite its computationally simple operations, relies on general-purpose integer instructions (rotations, shifts, additions) unless the processor supports dedicated SHA extensions.

Despite this performance difference, both primitives are extremely fast compared to RSA-based operations, confirming that symmetric cryptographic mechanisms are suitable for processing large volumes of data efficiently.

6.3 RSA-based Encryption vs. Decryption

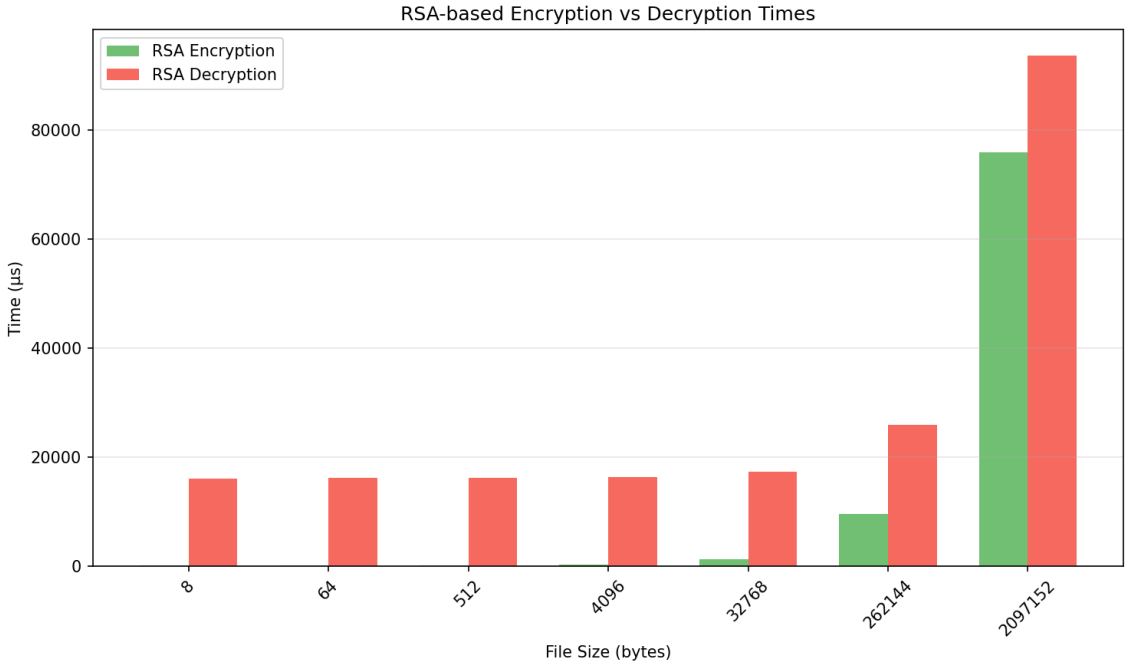


Figure 6: RSA-based encryption vs. decryption times.

Figure 6 highlights the pronounced asymmetry between RSA-based encryption and decryption. For the smallest file sizes (8–64 bytes), where the modular exponentiation dominates the total cost, decryption is roughly 140× slower than encryption (approximately 16,000 μs versus 114 μs). This ratio is a direct consequence of the exponent sizes: the public exponent $e = 65537 = 2^{16} + 1$ has only 17 bits, requiring approximately 17 modular squarings and one multiplication via the square-and-multiply algorithm. In contrast, the private exponent d is approximately the same size as the modulus n (2048 bits), requiring roughly 2048 modular squarings and, on average, 1024 additional multiplications.

As file size increases, the linear per-block component (SHA-256 hashing and XOR) becomes increasingly significant for both operations. Since this component is identical for encryption and decryption, the absolute gap between the two curves remains roughly constant while the relative gap narrows. At 2 MB, encryption takes approximately 76,000 μs and decryption approximately 93,000 μs, reducing the ratio to approximately 1.2x×.

It is worth noting that our implementation performs RSA decryption as a single `pow(c, d, n)` call, without employing the Chinese Remainder Theorem (CRT). A CRT-based implementation would split the computation into two smaller exponentiations modulo the prime factors p and q, potentially reducing the decryption time by a factor of approximately 4×. The decryption times reported here therefore represent an upper bound on what an optimized RSA implementation would achieve.

6.4 Overall Comparison

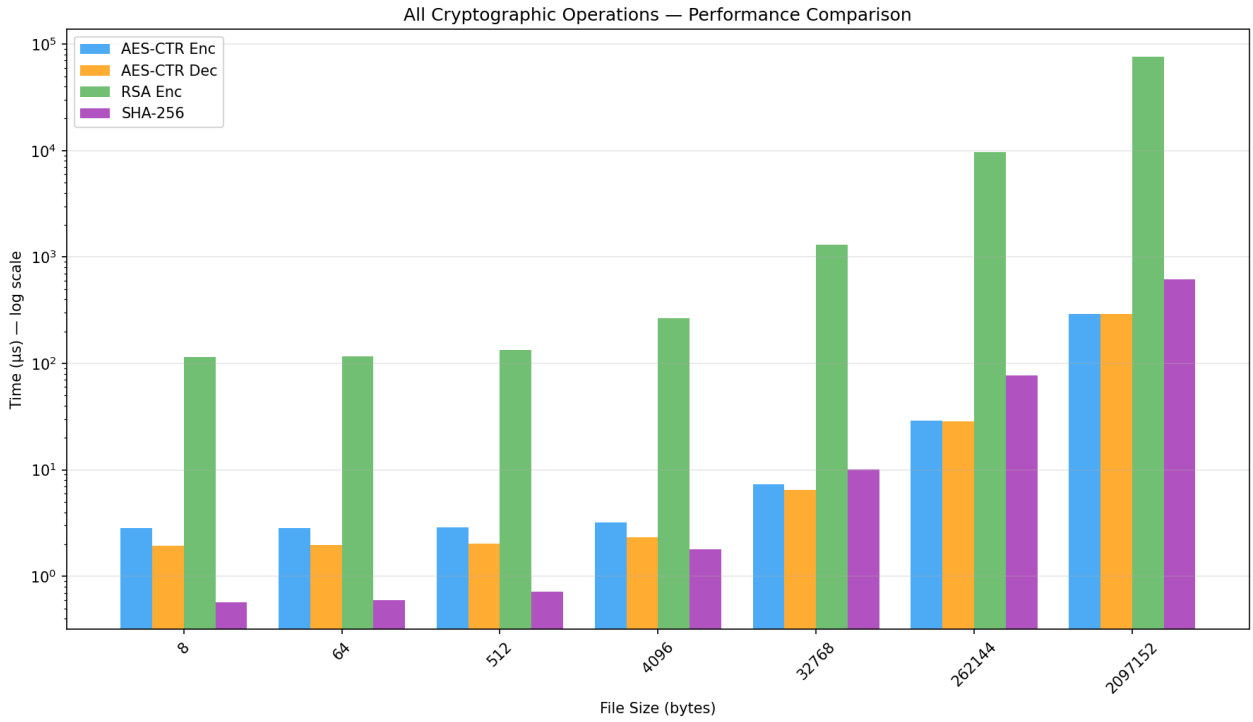


Figure 7: All cryptographic operations compared (log scale).

Figure 7 consolidates all five cryptographic operations on a single logarithmic plot, providing a comprehensive view of the performance landscape. The results establish a clear hierarchy across all tested file sizes:

$$t_{\text{SHA-256}} \approx t_{\text{AES-CTR}} \ll t_{\text{RSA enc}} \ll t_{\text{RSA dec}}$$

For small files, SHA-256 and AES-CTR are sub-microsecond operations, RSA encryption costs hundreds of microseconds, and RSA decryption requires tens of milliseconds — spanning roughly four orders of magnitude. For the largest file (2 MB), SHA-256 and AES-CTR remain under 1 ms, while RSA encryption and decryption reach tens to hundreds of milliseconds.

These results have direct implications for the design of secure communication systems. In protocols such as TLS 1.3, a *hybrid encryption* approach is employed: an asymmetric algorithm (RSA or, more commonly today, Elliptic Curve Diffie–Hellman) is used only once at the start of a session to establish a shared symmetric key, and all subsequent bulk data encryption is performed with AES (or ChaCha20). This design exploits the key-distribution capabilities of asymmetric cryptography while avoiding its prohibitive performance cost for large data volumes. Similarly, SHA-256 is extensively used for message authentication (HMAC), digital signature hashing, and integrity verification precisely because its performance scales efficiently with data size. Our benchmarking results quantitatively validate this architectural pattern: the performance gap between symmetric and asymmetric primitives is not merely theoretical but spans multiple orders of magnitude in practice.

7 Conclusion

This report presented a systematic performance benchmarking of three fundamental cryptographic mechanisms — AES-256-CTR symmetric encryption, an RSA-2048-based asymmetric encryption scheme, and SHA-256 cryptographic hashing — across file sizes spanning from 8 bytes to 2 MB.

Our measurements confirm the well-known performance hierarchy: symmetric primitives (AES-CTR and SHA-256) are orders of magnitude faster than asymmetric operations (RSA). AES-256-CTR encryption and decryption are computationally equivalent due to the symmetric nature of CTR mode and exhibit linear scaling with file size, completing in under 300 μs even for 2 MB files. SHA-256 displays similar linear scaling, with execution times dominated by the number of 64-byte blocks processed. The

RSA-based scheme, by contrast, incurs a substantial fixed cost due to modular exponentiation on 2048-bit integers, with decryption being significantly slower than encryption owing to the large private exponent.

These empirical results provide quantitative justification for the hybrid encryption paradigm used in modern security protocols such as TLS, PGP, and S/MIME: asymmetric cryptography is employed for key establishment (where only small values are encrypted), while symmetric ciphers handle bulk data encryption at speeds that are practical for real-time communication. The benchmarking methodology — combining multiple random files, adaptive repetitions, and rigorous statistical reporting — proved effective in producing stable, reproducible measurements with narrow confidence intervals, demonstrating that careful experimental design is essential for meaningful performance evaluation of cryptographic implementations.

AI Usage Disclosure

In the preparation of this report and the accompanying practical assignment, Artificial Intelligence tools (specifically Claude by Anthropic) were utilized strictly as auxiliary aids to support the authors' independent work. The primary application of these tools was to assist in refining the structural organization and formatting of the LaTeX document, ensuring a clean and professional presentation.

Furthermore, AI was leveraged as a supplementary educational resource during the research and study phases to help clarify technical concepts and explore standard benchmarking methodologies. Regarding the software implementation, AI assistance was highly restricted; it was employed solely for minor debugging purposes and to consult on specific, isolated code segments when troubleshooting errors.

The core architecture of the benchmarking framework, the data generation strategies, and the entire experimental setup were developed independently. All code implementations were rigorously reviewed, tested, and validated by the authors to ensure correctness and security. The final performance analysis, statistical evaluation, and critical interpretation of the cryptographic benchmarks represent the original intellectual effort and conclusions of the authors.